Operating Systems — 60 MCQs

Process Management • CPU Scheduling • Deadlocks & Synchronization • IPC & Signals • System Calls & Library Calls

Difficulty distribution: 12 Easy (20%) · 36 Medium (60%) · 12 Hard (20%). Four options per question; one best answer. Brief explanations are included in the answer key.

Instructions: Attempt the questions before checking the answer key. Scheduling questions use only the assumptions stated in each question.

Easy Difficulty — Questions 1–12

Q1. Which data structure stores process state, PID, saved registers and scheduling information?

A. PCB

B. TLB

C. ALU

D. FIFO

Q2. Which Linux function returns the caller's process ID?

A. getppid()

B. getpid()

C. wait()

D. exec()

Q3. On success, what does fork() return in the child?

A. Child PID

B. -1

C. 0

D. Parent PID

Q4. Which call replaces the current process image on success?

A. wait()

B. exec() family

C. getpid()

D. pipe()

Q5. A process waiting for I/O completion is normally in which state?

A. Running

B. Ready

C. New

D. Waiting/blocked

Q6. What is the main purpose of wait()/waitpid()?

A. Create a thread

B. Wait for a child and collect status

C. Replace a program image

D. Allocate memory

Q7. Which algorithm schedules in order of arrival?

A. FCFS

B. SJF

C. Round Robin

D. Priority

Q8. Which algorithm gives each ready process a time quantum?

A. FCFS

B. Round Robin

C. Non-preemptive SJF

D. Priority only

Q9. Mutual exclusion means:

A. Everyone enters together

B. No process ever blocks

C. At most one enters a protected section at a time

D. All processes have equal priority

Q10. Which is a classic necessary deadlock condition?

A. Circular wait

B. Time quantum

C. Aging

D. Paging

Q11. Which IPC mechanism is a byte stream often used between related processes?

A. PCB

B. Pipe

C. TLB

D. Mutex only

Q12. Which is a C library output function?

A. fork()

B. waitpid()

C. printf()

D. execve()

Medium Difficulty — Questions 13–48

Q13. What does fork() return in the parent on success?

A. 0

B. Child PID

C. -1 always

D. Parent PID

Q14. A terminated child not yet reaped by its parent is called a:

A. Orphan

B. Zombie

C. Daemon

D. Dispatcher

Q15. A child continues running after its parent terminates. It is commonly called:

A. Zombie

B. Ready

C. Orphan

D. Critical

Q16. What happens after a successful exec()?

A. A second PID is always created

B. The calling process image is replaced

C. All children terminate

D. The call returns to old code

Q17. What is typically saved/restored during a context switch?

A. Only source code

B. CPU registers and program counter

C. Entire disk

D. Only process name

Q18. Which statement about threads is generally correct?

A. Threads in a process share its address space

B. Every thread has an independent address space

C. Threads never need synchronization

D. Processes cannot communicate

Q19. CT=20 ms, AT=2 ms, BT=7 ms. What is waiting time in the single-burst model?

A. 11 ms

B. 18 ms

C. 13 ms

D. 9 ms

Q20. All arrive at 0: P1=6, P2=8, P3=7, P4=3. Non-preemptive SJF order?

A. P1,P2,P3,P4

B. P4,P1,P3,P2

C. P2,P3,P1,P4

D. P4,P3,P2,P1

Q21. An extremely small Round Robin quantum tends to cause:

A. No context switches

B. More context-switch overhead

C. Non-preemptive SJF

D. No scheduling overhead

Q22. Which is preemptive by its usual definition?

A. Round Robin

B. FCFS

C. Non-preemptive SJF

D. Run-to-completion

Q23. Indefinite postponement because higher-priority processes keep running is:

A. Aging

B. Starvation

C. Progress

D. Mutual exclusion

Q24. Which technique can reduce priority starvation?

A. Aging

B. Increase burst time

C. Remove ready queue

D. Disable interrupts

Q25. Dispatch latency is:

A. Time to switch from one process to another

B. Disk capacity

C. Compilation time

D. Jobs completed per second

Q26. A race condition occurs when:

A. A program uses a loop

B. Outcome depends on concurrent operation order

C. A process has a PID

D. Paging is enabled

Q27. Classic critical-section requirements include:

A. Throughput, paging, swapping

B. Mutual exclusion, progress, bounded waiting

C. PID, PPID, UID

D. Send, receive, close

Q28. Semaphore wait (P/down) typically:

A. Always increments

B. Attempts to acquire/decrement and may block

C. Creates a process

D. Closes a pipe

Q29. Semaphore signal (V/up) typically:

A. Makes a resource/condition available

B. Terminates the kernel

C. Replaces the process image

D. Guarantees no deadlock

Q30. Which is NOT a classic necessary deadlock condition?

A. Mutual exclusion

B. Hold and wait

C. Circular wait

D. Time quantum expiration

Q31. Two threads acquire mutexes A then B and B then A. Main risk?

A. Guaranteed progress

B. Circular wait/deadlock

C. Paging

D. New PID

Q32. Test-and-Set is used to:

A. Perform an atomic operation for synchronization

B. Allocate a file

C. Calculate turnaround

D. Send packets

Q33. Best IPC for high-volume shared data on one machine, with synchronization?

A. Shared memory

B. SIGINT only

C. PID printing

D. CPU scheduling

Q34. Signals are mainly used to:

A. Transfer arbitrary bulk data

B. Notify a process of an event

C. Replace pipes in every case

D. Run only synchronously

Q35. For pipe(fd), fd[0] is normally the:

A. Write end

B. Read end

C. PID

D. Exit status

Q36. What should a process usually do with an unused pipe endpoint?

A. Keep it open forever

B. Close it

C. Convert it to a PID

D. Use it as a signal

Q37. Message passing commonly provides which operations?

A. open/close

B. send/receive

C. fork/exit only

D. bind/listen only

Q38. Cooperating processes are processes that:

A. Can affect or be affected by one another

B. Must share one PID

C. Never exchange information

D. Cannot be scheduled

Q39. Which is a POSIX system-call interface?

A. printf()

B. fopen()

C. read()

D. strlen()

Q40. Which statement about printf() is accurate?

A. It is the scheduler

B. It is a C library formatting function

C. It creates a child

D. It always maps one-to-one to a syscall

Q41. For a regular file, read() returning 0 usually means:

A. End of file

B. New child PID

C. Deadlock

D. Successful exec

Q42. The primary purpose of a system call is to:

A. Request a kernel service

B. Compile source

C. Define CPU burst

D. Replace mutex

Q43. waitpid(pid,&status;,0) normally:

A. Waits for that child and collects status

B. Replaces parent program

C. Creates a thread

D. Sets priority

Q44. After successful exec, which statement is true?

A. Old code immediately continues

B. Process image replaced; PID normally remains same

C. Parent PID becomes zero

D. All descriptors always close

Q45. Why use a pipe between parent and child?

A. Transfer a byte stream

B. Change child PID

C. Force FCFS

D. Eliminate all deadlocks

Q46. Which best describes a zombie?

A. Always consumes a CPU core

B. Terminated but not yet reaped

C. A thread sharing memory

D. A process waiting for I/O

Q47. Ready vs waiting state: which is correct?

A. Ready waits for CPU; waiting waits for event/resource

B. Ready is terminated

C. They are identical

D. Waiting owns CPU

Q48. Which statement about negative semaphore values is safest?

A. Every API exposes negative values

B. Some textbook conventions use them to represent waiters; definitions vary

C. It means negative PID

D. It proves deadlock

Hard Difficulty — Questions 49–60

Q49. Three processes arrive at 0 with bursts P1=5, P2=3, P3=1. Average waiting time under non-preemptive SJF?

A. 1.67 ms

B. 2 ms

C. 3 ms

D. 4 ms

Q50. RR quantum=2 ms; all arrive at 0; P1=4, P2=3. When does P2 complete, ignoring overhead?

A. 3 ms

B. 5 ms

C. 7 ms

D. 4 ms

Q51. CT=30 ms, AT=4 ms, BT=10 ms. TAT and WT?

A. 26 ms, 16 ms

B. 30 ms, 20 ms

C. 26 ms, 10 ms

D. 34 ms, 24 ms

Q52. Binary semaphore S=1. T1 waits and holds it. T2 calls wait(S). What happens?

A. T2 also enters

B. T2 cannot acquire until released

C. S creates child

D. T2 resets S

Q53. Two threads increment x with separate read/add/write; both read 10 and write 11. Possible final x?

A. 12 only

B. 10 only

C. 11

D. 20

Q54. Which change directly prevents circular wait for locks?

A. Require a single global lock-acquisition order

B. Increase quantum

C. Add print statements

D. Replace PID with PPID

Q55. Parent forks child; child successfully execs; parent waits. What does waitpid wait for?

A. Old child code to return from exec

B. That same child process to change state/terminate

C. A new PID from exec

D. Child to become parent

Q56. A process writes to a full pipe and no reader drains it. What is a key risk?

A. Blocking/stall due to full pipe buffer

B. Automatic SJF

C. Guaranteed EOF

D. PID overflow

Q57. Which statement about the four deadlock conditions is correct?

A. Each alone guarantees deadlock

B. All four are necessary in the classic model

C. Starvation is one of them

D. Time slicing is one

Q58. What is safest in an asynchronous signal handler?

A. Call any complex library function

B. Do minimal async-signal-safe work

C. Call printf repeatedly without care

D. Acquire arbitrary mutexes

Q59. fopen() and fclose() are best described as:

A. Direct system calls only

B. C library interfaces that use lower-level OS services

C. Process creation calls

D. Scheduling algorithms

Q60. A counting semaphore starts at 3 for three identical resources. Maximum simultaneous holders?

A. 1

B. 2

C. 3

D. 4

Answer Key and Explanations

Q1. A — The Process Control Block stores information used by the OS to manage a process.

Q2. B — getpid() returns the current PID; getppid() returns the parent PID.

Q3. C — fork() returns 0 in the child and the child PID in the parent.

Q4. B — Successful exec replaces the program image and does not return to old code.

Q5. D — It is blocked until the relevant event completes.

Q6. B — Wait functions collect child state or termination information.

Q7. A — First-Come, First-Served follows arrival order.

Q8. B — Round Robin rotates through ready processes using time slices.

Q9. C — Mutual exclusion prevents concurrent entry into the same protected critical section.

Q10. A — Circular wait is one of the four classic conditions.

Q11. B — Pipes commonly transfer data between related processes.

Q12. C — printf formats output and may eventually use a lower-level write operation.

Q13. B — The parent receives the child's PID.

Q14. B — Its process-table entry remains until its status is collected.

Q15. C — An orphan is re-parented by the system.

Q16. B — exec changes the program image but normally preserves the process PID.

Q17. B — The OS preserves the execution context.

Q18. A — Threads share many process resources, so shared data requires coordination.

Q19. A — TAT=20−2=18; WT=18−7=11 ms.

Q20. B — Choose bursts in ascending order: 3, 6, 7, 8.

Q21. B — Frequent preemption can increase overhead.

Q22. A — Round Robin preempts when a time quantum expires.

Q23. B — Starvation is indefinite or excessive waiting for CPU/resources.

Q24. A — Aging gradually improves priority for waiting processes.

Q25. A — It is the time needed to stop one process and start another.

Q26. B — Unsynchronised interleavings can change the result.

Q27. B — These are the standard correctness requirements.

Q28. B — Wait can block when the resource or condition is unavailable.

Q29. A — Signal releases/increments according to semaphore semantics.

Q30. D — The four are mutual exclusion, hold-and-wait, no preemption and circular wait.

Q31. B — Inconsistent lock ordering can create circular wait.

Q32. A — It is an indivisible primitive that can help implement locks.

Q33. A — Shared memory avoids repeated message copying but requires coordination.

Q34. B — Signals provide event notification, not general bulk-data transport.

Q35. B — POSIX pipe convention: fd[0] reads and fd[1] writes.

Q36. B — Closing unused ends supports correct EOF and broken-pipe behaviour.

Q37. B — Send and receive are the basic message-passing operations.

Q38. A — They coordinate or exchange data.

Q39. C — read() is a system-call interface; the others are library functions.

Q40. B — It formats output and may call lower-level I/O.

Q41. A — A zero-byte read normally indicates EOF.

Q42. A — System calls provide controlled access to kernel services.

Q43. A — It can wait for a specified child.

Q44. B — exec preserves process identity; descriptors close only if marked close-on-exec.

Q45. A — A pipe is a byte-stream IPC channel.

Q46. B — A zombie has exited but its status has not been collected.

Q47. A — Ready processes can be scheduled; blocked processes need an event.

Q48. B — Interpretation depends on semaphore convention and implementation.

Q49. A — Order P3,P2,P1; waits 0,1,4; average=5/3≈1.67 ms.

Q50. C — P1 0–2, P2 2–4, P1 4–6, P2 6–7.

Q51. A — TAT=30−4=26; WT=26−10=16 ms.

Q52. B — Only one thread can hold the binary semaphore at a time.

Q53. C — One increment is lost when both write 11.

Q54. A — Consistent lock ordering prevents cycles in lock acquisition.

Q55. B — exec replaces the child image but not its process identity.

Q56. A — A write can block when the pipe buffer is full.

Q57. B — The classic model requires all four conditions to hold for deadlock.

Q58. B — Many library functions are not async-signal-safe.

Q59. B — They are standard C library interfaces.

Q60. C — At most three successful acquisitions occur before releases.